TARA-16 Microprogrammed CPU in Logisim-Evolution

Final project report - CS2310 take-home lab (TH1-TH6)

Roll number: **CS25B035** | Group 38 (individual) | Logisim-Evolution 4.1.0

Summary. A complete 16-bit processor for the TARA instruction set was built in Logisim-Evolution: a single-bus datapath (TH1-TH4), a microprogrammed control unit driving the fifteen core instructions (TH5), and memory, byte, branch, subroutine and stack instructions (TH6). The control unit is a 4-bit T-state counter plus a 2048 x 32 control ROM addressed by {Z, N, OPCODE[4:0], TSTATE[3:0]}; its contents are generated and checked by a Python microcode generator. All 26 graded instructions are implemented (MUL and undefined opcodes halt by design). Every one of the 27 test programs - 21 unit tests and 6 integration programs - finishes with registers, PC and all 2048 bytes of memory identical to the reference simulator TARA Studio.

Item	Result
Graded instructions implemented	26 / 26 (15 core in TH5, 11 in TH6)
Test programs identical to TARA Studio	27 / 27 (TH5 suite 13 / 13)
Microcode generator consistency checks	7 / 7 (TH5 and TH6)
Control ROM vs generator output	2048 / 2048 words identical
Fetch	3 clocks; instructions take 4-7 clocks in total

1. How this report is organised

Sections 1-7 give an overview of the whole processor and its verification. **Appendix A** is the full TH5 report (control unit, core instructions, interface table, microprograms). **Appendix B** is the full TH6 report (memory, branches, CALL/RET, PUSH/POP, integration programs). Source files referenced: `th6/G38_th6_tara.circ`, `th6/G38_th6_microcode_gen.py`, `th6/G38_th6_control_rom.hex`, `tools/make_images.py`, test material in `th5/tests` and `th6/tests`.

2. Project stages

Stage	What was built
TH1-TH3	16-bit registers, 8 x 16 register file with two read ports, ALU (ADD SUB AND OR XOR NOT SHL SHR SLT) with Z N C V flags, instruction decoder
TH4	Single-bus datapath: PC/MAR/MDR/IR, tri-state bus drivers, memory, manual switch panel for every control signal
TH5	Microprogrammed control unit (T-state counter, 2048 x 32 control ROM, HALT latch); imm_unit with 8 immediate forms; ALU operand muxes; status register; asynchronous RAM read; 15 core instructions
TH6	Byte lanes in ram_word (BYTE_MODE), register-file write-index mux (R6/R7), 32-bit control-word re-layout; LDW STW LDB STB JMP BZ BN CALL RET PUSH POP; six integration programs

3. Architecture

All transfers use one 16-bit shared bus. Exactly one of PC_OUT, MDR_OUT, IMM_OUT, RF_OUT, ALU_OUT, RAM_OUT_LD drives it in any clock; LOAD/WRITE signals capture it on the rising edge. The ALU takes its A input from register port A or the PC (ALU_A_SEL) and its B input from port B or the immediate unit (ALU_B_SEL), so effective addresses and PC-relative targets are computed without extra adders. Memory is big-endian and byte addressed: ram_word holds two 8-bit RAMs (bottom = high byte at the even address, top = low byte), indexed by addr[15:1].

Subcircuit	Role
tara_cpu (main)	clock, reset, 32-bit control-word splitter
control_unit	T-state counter, control ROM, HALTED flip-flop + mux forcing cw = 0 after HLT
tara_datapath	bus, tri-state drivers, RA index mux, RF write-index mux, ALU operand muxes
PC_MAR_MDR_IR	PC (PC+2 or load from bus), MAR, MDR, IR
tara_decoder	IR -> opcode, RD, RA, RB, imm8, off5, rel8, rel11
imm_unit	0 zext8, 1 sext8, 2 sext5, 3 rel8x2, 4 rel11x2, 5 const 2, 6 {imm8, QA[7:0]} (LIH), 7 zero
regfile8x16	R0-R7, two read ports (qa, qb), one write port
ALU / status_reg	operation selected by ALU_select; Z N C V latched when status_we = 1
ram_word	two byte RAMs; BYTE_MODE: per-bank write enable, zero-extended byte read

4. Control unit and microcode

On each clock the ROM address {Z, N, opcode, T} selects one 32-bit control word. T0-T2 are the same for all opcodes (fetch); from T3 the opcode block of the ROM supplies the execute steps, and the last step sets STEP_RESET so the

counter returns to T0. Conditional branches need no comparator in the control unit: BZ/BN set the flags from Rt at T3, and the latched Z or N bit in the ROM address selects either the "taken" (PC <- PC + rel x 2) or the "not taken" (STEP_RESET) word at T4. The generator writes all 2048 words and refuses to output a ROM unless 7 checks pass (single bus driver, every instruction terminates, branch words differ only in the flag bit, unimplemented opcodes halt, etc.).

Bits	Field	Bits	Field	Bits	Field
0	PC_OUT	8	IR_LOAD	21	status_we
1	MDR_OUT	9	RF_WRITE	22-24	IMM_SEL
2	IMM_OUT	10	PC_EN	25-26	RA_SEL
3	RF_OUT	11	PC_LOAD	27-28	RF_IDX_SEL
4	ALU_OUT	12	mem_read	29	STEP_RESET
5	RAM_OUT_LD	13	mem_write	30	HALT
6	MAR_LOAD	14-18	ALU_select	31	BYTE_MODE
7	MDR_LOAD	19 / 20	ALU_A_SEL / ALU_B_SEL		

Example - `LDW R1, 0(R2)` (IR = 0x2940, opcode 5):

T	ROM addr	Word	Action
T0	0x050	0x00000041	PC -> MAR
T1	0x051	0x000014A0	M[MAR] -> MDR, PC <- PC + 2
T2	0x052	0x00000102	MDR -> IR
T3	0x053	0x00924050	MAR <- R2 + sext5(0) (ALU ADD, B = imm_unit form 2)
T4	0x054	0x000010A0	MDR <- M[MAR]
T5	0x055	0x20000202	R1 <- MDR, STEP_RESET

5. Toolflow: from a .tara program to a result

Step	Tool	Output
1. Build the microcode (once)	G38_th6_microcode_gen.py	G38_th6_control_rom.hex, listing, check log -> loaded into the control ROM
2. Assemble the program	tools/make_images.py (TARA Studio assembler)	16-bit words; split into _HIGH.hex (bottom RAM) and _LOW.hex (top RAM); expected registers printed
3. Load and reset	Logisim	PC = 0, T = 0, HALTED = 0; program in ram_word
4. Run	Logisim clock	fetch / execute until HLT sets HALTED and freezes the CPU
5. Check	regfile8x16, RAM contents	compare R0-R7, PC, memory with TARA Studio

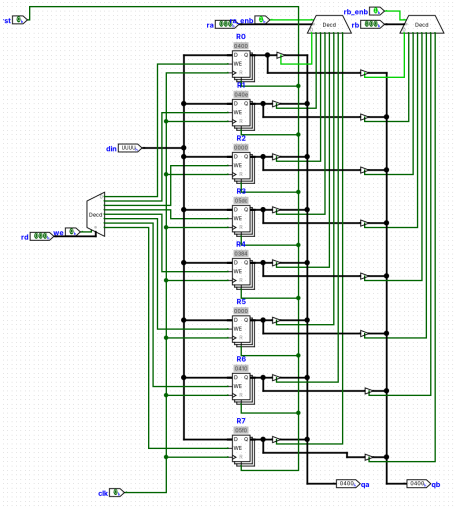
6. Verification

Each program was assembled by TARA Studio's assembler, loaded into ram_word and run on the circuit with Logisim's own simulator until HALTED = 1; the same program was run on TARA Studio's CPU model. PASS means R0-R7, PC and all 2048 bytes of memory are identical (t21: against the documented MUL-halts policy). Per-instruction traces of the six integration programs on the circuit are in th6/tests/traces.

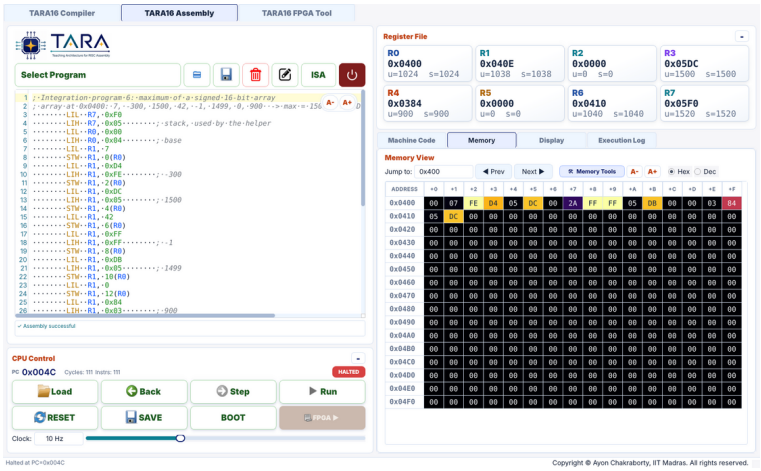
Program	Result	R0 - R7 (final)	PC
t01_fetch_nop_hlt	PASS	`0000 0000 0000 0000 0000 0000 0000 0000`	000A
t02_lil	PASS	`00FF 0080 0000 0000 0000 0000 0000 002A`	000A
t03_lih	PASS	`ABCD FF34 0500 00FF 0000 0000 0000 0000`	0012
t04_mov	PASS	`0000 1234 1234 0000 0000 0000 0000 1234`	000E
t05_add	PASS	`0000 7FFF 0002 8000 FFFF 0000 0002 0000`	0014
t06_sub	PASS	`0000 0003 0005 FFFE 0002 8000 0001 7FFF`	0014
t07_addi	PASS	`0000 0008 FFFF 8000 007F 0000 0000 0000`	0016
t08_logic	PASS	`FFFF 0FF0 A53C 0530 AFFC AACC 0000 F00F`	0016
t09_shift	PASS	`0000 0028 8000 0000 4000 0001 00FF 7FFF`	0024
t10_slt	PASS	`0001 FFFF 0001 0001 0000 0000 0000 7FFF`	001A
t11_ldw_stw	PASS	`0000 0410 1234 1234 0410 1234 0000 0410`	001C
t12_ldb_stb	PASS	`00AB 0400 1234 0012 0034 99EE 12EE EE00`	0022
t13_jmp	PASS	`0000 0001 0002 0003 0000 0000 0000 0000`	0012
t14_bz	PASS	`0000 0000 FFFF 0000 0000 0004 0000 0000`	0024
t15_bn	PASS	`FFFF 0005 8000 0000 0003 0000 0000 0000`	001E
t16_call_ret	PASS	`0000 0052 0000 0000 0000 0014 000C 05F0`	000E
t17_push_pop	PASS	`05F0 0011 AA22 AA22 0011 05EE 05EE 0013`	0020
t18_pop_underflow	PASS	`0000 0077 0077 0800 0000 0000 0000 0800`	0012
t19_regression_mix	PASS	`0406 0000 0015 002A 002A FFC0 002A 05E6`	002C
t20_regfile_ports	PASS	`FEE1 FFEF FFEF FFEF FFEF FFEF FFEF 010E`	0026
t21_unimplemented_mul	PASS	`0000 0003 0000 0000 0000 0000 0000 0000`	0004

Program	Result	R0 - R7 (final)	PC
p1_register_addition	PASS	0000 1234 0FED 2221 0000 0000 0000 0000	000C
p2_memory_subtraction	PASS	0400 012C 07D0 012C 06A4 0000 0000 0000	001A
p3_parity	PASS	0001 0000 0000 0001 0000 B6D5 0000 0000	001A
p4_multiply_repeated_addition	PASS	0000 007B 0000 159F 0000 0000 0000 0000	0010
p5_array_sum	PASS	0400 0410 0000 1194 0000 007D 0000 0000	0030
p6_array_max	PASS	0400 040E 0000 05DC 0384 0000 0410 05F0	004C

27 / 27 PASS.



Logisim: register file after p6_array_max



TARA Studio after p6_array_max - identical registers

7. Problems found and fixed during development

Stage	Problem	Fix
TH4	R7 wire to read port B 10 units short; qa / qb shorted; ra_enb / rb_enb floating	wire completed; outputs separated; enables tied
TH4	no ACC and no status register; only a zero-extended immediate	ACC, status_reg and imm_unit (8 forms) added
TH4	ALU opcode input unconnected; operands passed through controlled inverters	opcode driven from ALU_select; inverters removed
TH4	synchronous RAM read gave data one clock late	RAM set to asynchronous read
TH5	RA index mux select unconnected	RA_SEL pin added to the datapath and control word
TH5	T-state adder fed by a stray input pin -> counter stuck at T1	stray pin removed; adder fed from the T-state register
TH5	imm_unit QA and rel11 pins off-grid from their tunnels	pins moved onto the tunnels
TH6	no way to write R6 / R7 for CALL, PUSH, POP	RF_IDX_SEL write-index mux (RD, 6, 7)
TH6	control word full (32 bits)	unused IR_OUT tied to 0, ACC_load tied to ALU_OUT; freed bits used for RF_IDX_SEL, BYTE_MODE

8. Limitations

MUL (opcode 12) and undefined opcodes 27-31 halt without changing state. Word accesses ignore addr[0] (aligned only). ram_word has 32 K words, so addresses above 0x7FF are not wrapped as in TARA's 2 KB memory; no program uses them.

Appendix A - TH5 report (4 pages) **Appendix B** - TH6 report (5 pages)

CS2310 Take-home 5: Control unit and core instruction set

Group 38 | Roll number: **CS25B035** | Logisim-Evolution 4.1.0 | G38_th5_control.circ, G38_th5_microcode_gen.py, G38_th5_control_rom.hex, G38_th5_tests/

Summary. The manual switch panel of our TH4 datapath is driven by a microprogrammed control unit: a 4-bit T-state register and a 2048 x 32 control ROM addressed by {Z, N, OPCODE[4:0], TSTATE[3:0]}, with Z and N taken from the latched outputs of status_reg. The ROM image is produced by G38_th5_microcode_gen.py, which passes all its consistency checks (7/7). All fifteen core instructions are implemented in microcode and every test program, including the boundary cases, ends with registers and PC identical to TARA Studio (13/13). **Condition A** (four consecutive fetches under ROM control) and **Condition B** (programmatically generated ROM) are both met. Core pool claimed: 43/43.

1. Interface table

New or changed in TH5 (all ports, exactly as labelled in the file):

Subcircuit	Inputs	Outputs	Status
tara_cpu (main)	RST; internal Clock component CLK	(none; values are observed on the subcircuit pins)	new
control_unit	CLK, RST, Z, N, opcode[5]	cw[32], T_STATE[4], HALTED, HALT	new
tara_datapath	CLK, RST, PC_OUT, IR_OUT, MDR_OUT, IMM_OUT, RF_OUT, ALU_OUT, RAM_OUT_LD, MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_load, RF_WRITE, PC_EN, PC_LOAD, mem_read, mem_write, ALU_select[5], ALU_A_SEL, ALU_B_SEL, status_we, IMM_SEL[3], RA_SEL[2]	opcode[5], Z, N, PC[16], IR[16]	changed: ALU operand muxes, RA index mux, imm_unit, ACC, status_reg, outputs to control unit
imm_unit	imm8[8], off5[5], rel8[8], rel11[11], QA[16], IMM_SEL[3]	IMM[16]	new
ram_word	addr[16], din[16], mem_read, mem_write, clk	dout[16]	changed: asynchronous read
regfile8x16	ra[3], rb[3], rd[3], din[16], we, clk, rst, ra_enb, rb_enb	qa[16], qb[16]	fixed (R7 to port B)

Reused unchanged from TH1-TH4:

Subcircuit	Ports	Subcircuit	Ports
ALU	A[16], B[16], OPCODE[5], ALU_OUTPUT_ENABLE → BUS_OUT[16], ZERO, NEGATIVE, CARRY, OVERFLOW	tara_decoder	IR[16] → op[5], format[3], RD, RA, RB[3], imm8[8], off5[5], rel8[8], rel11[11]
status_reg	Z_in, N_in, C_in, V_in, status_we, clk, RST → Z, N, C, V	PC_MAR_MDR_IR	CLK, RST, MAR_D/EN, MDR_D/EN, IR_D/EN, PC_D, PC_LOAD, PC_EN → PC_Q, MAR_Q, MDR_Q, IR_Q
register16	d[16], clk, en, rst → q[16] (used as ACC)	dff	D, CLK, EN, RST → Q
adders, addsub16	half/full adder, adder4, adder16, addsub16 (SUB, OVF)	shifters, comparators	SHL16/SHR16 (A, SH[5] → Y), shiftL/Rby1..16, cmp1/cmp4/cmp16
ALU internals	mux_tree, conditional_logic, Tri_state_buffer, op_to_format		

2. Control word (one field per datapath input pin; 29 of 32 bits used)

Bits	Field	Meaning when 1 / value	Bits	Field	Meaning when 1 / value
0	PC_OUT	PC → bus	12	PC_EN	PC update (with PC_LOAD=0: PC+2)
1	IR_OUT	IR → bus	13	PC_LOAD	with PC_EN: PC ← bus (TH6)
2	MDR_OUT	MDR → bus	14	mem_read	memory output enable
3	IMM_OUT	imm_unit → bus	15	mem_write	M[MAR] ← MDR (TH6)
4	RF_OUT	register port A (qa) → bus	16-20	ALU_select	ALU opcode (9 ADD ... 19 SLT)
5	ALU_OUT	ALU result → bus	21	ALU_A_SEL	ALU A: 0 qa, 1 PC
6	RAM_OUT_LD	memory → bus	22	ALU_B_SEL	ALU B: 0 qb, 1 IMM
7	MAR_LOAD	MAR ← bus	23	status_we	latch Z N C V
8	MDR_LOAD	MDR ← bus	24-26	IMM_SEL	0 zext8, 1 sext8, 2 sext5, 3 rel8x2, 4 rel11x2, 5 const 2, 6 {imm8,QA[7:0]}, 7 zero
9	IR_LOAD	IR ← bus	27-28	RA_SEL	port A reads 0 IR[7:5], 1 IR[10:8], 2 R6, 3 R7
10	ACC_load	ACC ← bus	29	STEP_RESET	next edge: T-state ← 0
11	RF_WRITE	R[IR[10:8]] ← bus	30	HALT	set HALTED; control word then forced to 0

3. Design decisions

Decision	Justification
Control word is the TH4 switch panel.	Each field is one input pin of tara_datapath with the same name, so the inventory from TH4 became the control word unchanged and the split in tara_cpu is a single 32-bit splitter. The seven bus sources keep separate enables; the course recommends an encoded field, and we compensate in the generator: check 1 rejects any word with two sources and check 4 rejects any load from an undriven bus.
ROM addressed by {Z, N, OPCODE, TSTATE}.	2048 x 32 words, Z and N from status_reg's latched outputs. In TH5 every word is identical for all four (Z, N) combinations (check 5); the duplication keeps the TH6 branches simple.
T-state counter and HALT.	A 4-bit register with a +1 adder; a 2:1 mux selects 0 when STEP_RESET (cw bit 29) is set. HALT (bit 30) sets a D flip-flop that holds itself through an OR gate; while HALTED=1 a 32-bit mux forces the control word to 0, so nothing can change until RST. PC has already advanced past HLT, as in TARA Studio.
Fetch in three states.	T0 0x00000081 (PC_OUT, MAR_LOAD); T1 0x00005140 (mem_read, RAM_OUT_LD, MDR_LOAD, PC_EN); T2 0x00000204 (MDR_OUT, IR_LOAD). The RAM reads asynchronously, so MDR captures M[MAR] at the end of T1 (see fault 2).
Every core instruction executes in one state (T3).	ALU operands come straight from the register-file ports (A = qa or PC, B = qb or IMM) and only the result uses the bus, so a register-register operation needs no temporary register. Each instruction takes 4 clock cycles.
RA index mux (RA_SEL).	LIH, ADDI, SHL and SHR operate on Rd, so read port A can be switched from IR[7:5] to IR[10:8]. Inputs 2 and 3 are the constants 6 and 7, already wired for RET and PUSH/POP in TH6.

Decision	Justification
LIH as read-modify-write in a single state.	Port A reads Rd, imm_unit forms {imm8, QA[7:0]} (IMM_SEL=6) and the result is written back to Rd, so the low byte is kept. Tested by building ABCD with LIL+LIH and by LIH over a non-zero low byte (t03).
ADDI sign-extends; LIL zero-extends.	IMM_SEL=1 (sext imm8) with RA_SEL=1 so Rd is both source and destination; LIL uses IMM_SEL=0. Tested with -1, -128, +127 and 7FFF+1 (t07).
SHL / SHR take the amount from the immediate.	ALU_B_SEL=1 selects imm_unit (zext imm8); the ALU uses B[4:0], so amounts 16-31 give 0 and 0x21 shifts by 1, exactly as TARA Studio (t09).
status_we on every ALU instruction; ACC loaded with every ALU result.	No core instruction reads Z/N, but latching them keeps the last condition visible on the probes and the field is ready for BZ/BN. ACC is an observation copy of the last ALU result.
Unimplemented opcodes HALT; unused ROM words step-reset.	MUL (12), opcodes 27-31 and the TH6 instructions get HALT at T3, so they cannot modify state (check 6, test t21). Every other unused address holds STEP_RESET, so a stray T-state only wastes one cycle.
Memory: big-endian, two byte banks.	ram_word keeps two 8-bit RAMs addressed by addr[15:1]; the top RAM holds the low byte and the bottom RAM the high byte, so M[even] is the high byte of the word, as in TARA Studio. A program is loaded as two images (_LOW / _HIGH).

4. Microprograms (T0-T2: common fetch; all end with STEP_RESET)

Instr.	opcode	T3 word	Signals	Effect
NOP	00000	0x20000000	STEP_RESET	-
HLT	00001	0x60000000	HALT, STEP_RESET	stop
MOV	00010	0x20000810	RF_OUT, RF_WRITE	Rd ← Rs
LIL	00011	0x20000808	IMM_SEL=0, IMM_OUT, RF_WRITE	Rd ← zext(imm8)
LIH	00100	0x2E000808	RA_SEL=1, IMM_SEL=6, IMM_OUT, RF_WRITE	Rd ← {imm8, Rd[7:0]}
ADD SUB AND OR XOR SLT	9 10 13 14 15 19	0x20890C20 ...	ALU_select, ALU_OUT, RF_WRITE, ACC_load, status_we	Rd ← Ra op Rb
NOT	10000	0x20900C20	ALU_select=16, ALU_OUT, RF_WRITE, ACC_load, status_we	Rd ← ~Rs
ADDI	01011	0x29C90C20	RA_SEL=1, ALU_B_SEL, IMM_SEL=1, ALU_select=9, ALU_OUT, RF_WRITE ...	Rd ← Rd + sext(imm8)
SHL / SHR	17 / 18	0x28D10C20 / 0x28D20C20	RA_SEL=1, ALU_B_SEL, IMM_SEL=0, ALU_OUT, RF_WRITE ...	Rd ← Rd <</>> imm8[4:0]
MUL, 27-31, TH6 ops		0x60000000	HALT, STEP_RESET	policy: halt

Generator log (G38_th5_microcode_log.txt), Condition B:

```
TARA microcode generator, TH5 — 15 instructions: NOP HLT MOV LIL LIH ADD SUB ADDI AND OR XOR NOT SHL SHR SLT
ROM 2048 x 32 bits

[PASS] 1. no control word selects more than one bus source
[PASS] 2. every instruction eventually asserts STEP_RESET or HALT
[PASS] 3. mem_read and mem_write are never asserted together
[PASS] 4. write enables are never asserted without valid source data
[PASS] 5. conditional-branch words differ only in the relevant condition signal
[PASS] 6. no undefined opcode can modify architectural state (they HALT)
[PASS] 7. T0-T2 hold the common fetch sequence for every opcode

7/7 checks passed
```

5. Test evidence

Method. Every program in G38_th5_tests/programs is assembled with the TARA Studio assembler, loaded into ram_word as two byte images, and run on our circuit until HALTED=1; the same program is run on TARA Studio's reference CPU model. All screenshots below are of our own circuit in Logisim after HLT. Full per-instruction reference logs are in G38_th5_tests/reference_traces.

Program	Instructions tested	R0 ... R7 after HLT (ours = TARA Studio)	PC	
t01	fetch x4, NOP, HLT	0000 0000 0000 0000 0000 0000 0000 0000	000A	PASS
t02	LIL	00FF 0080 0000 0000 0000 0000 0000 002A	000A	PASS
t03	LIH (low byte kept)	ABCD FF34 0500 00FF 0000 0000 0000 0000	0012	PASS
t04	MOV	0000 1234 1234 0000 0000 0000 0000 1234	000E	PASS
t05	ADD, 7FFF+1, FFFF+1	0000 7FFF 0002 8000 FFFF 0000 0002 0000	0014	PASS
t06	SUB, 3-5, 8000-1	0000 0003 0005 FFFE 0002 8000 0001 7FFF	0014	PASS
t07	ADDI -1, +127, -128, 7FFF+1	0000 0008 FFFF 8000 007F 0000 0000 0000	0016	PASS
t08	AND OR XOR NOT	FFFF 0FF0 A53C 0530 AFFC AACC 0000 F00F	0016	PASS
t09	SHL/SHR by 0,1,3,15,16,33	0000 0028 8000 0000 4000 0001 00FF 7FFF	0024	PASS
t10	SLT signed, equal, 8000 vs 7FFF	0001 FFFF 0001 0001 0000 0000 0000 7FFF	001A	PASS
t20	every register on ports A and B	FEE1 FFEF FFEF FFEF FFEF FFEF FFEF 010E	0026	PASS
t21	MUL → HALT, nothing changed	0000 0003 0000 0000 0000 0000 0000 0000	0004	PASS*
p1	register addition (regression)	0000 1234 0FED 2221 0000 0000 0000 0000	000C	PASS

* t21 is checked against our documented policy: TARA Studio executes MUL, our processor halts at it with no state change.

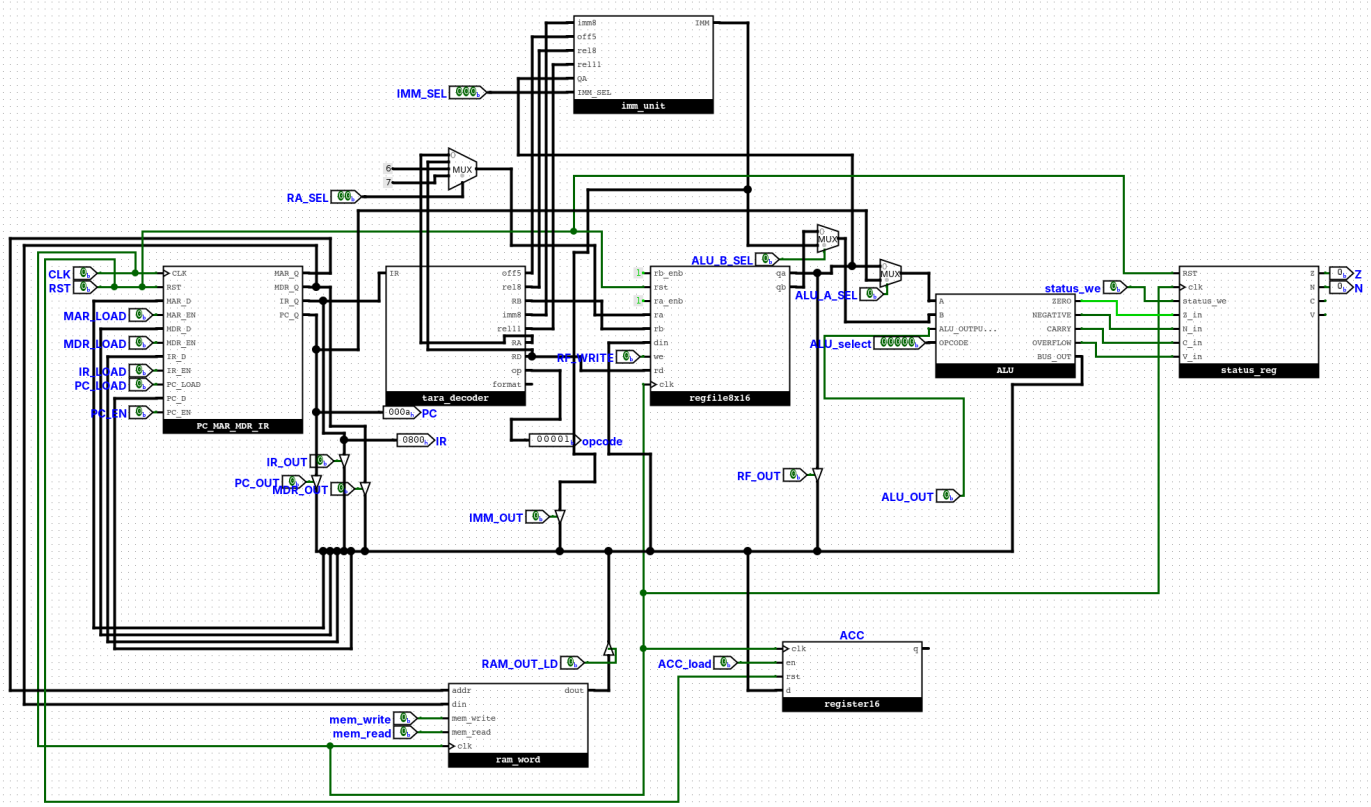


Figure 1. Condition A: tara_datapath after t01 (NOP x4, HLT). Four fetches completed under ROM control: PC = 000A, IR = 0800 (HLT), opcode = 00001. Control inputs are all 0 because the halted control unit forces cw = 0.

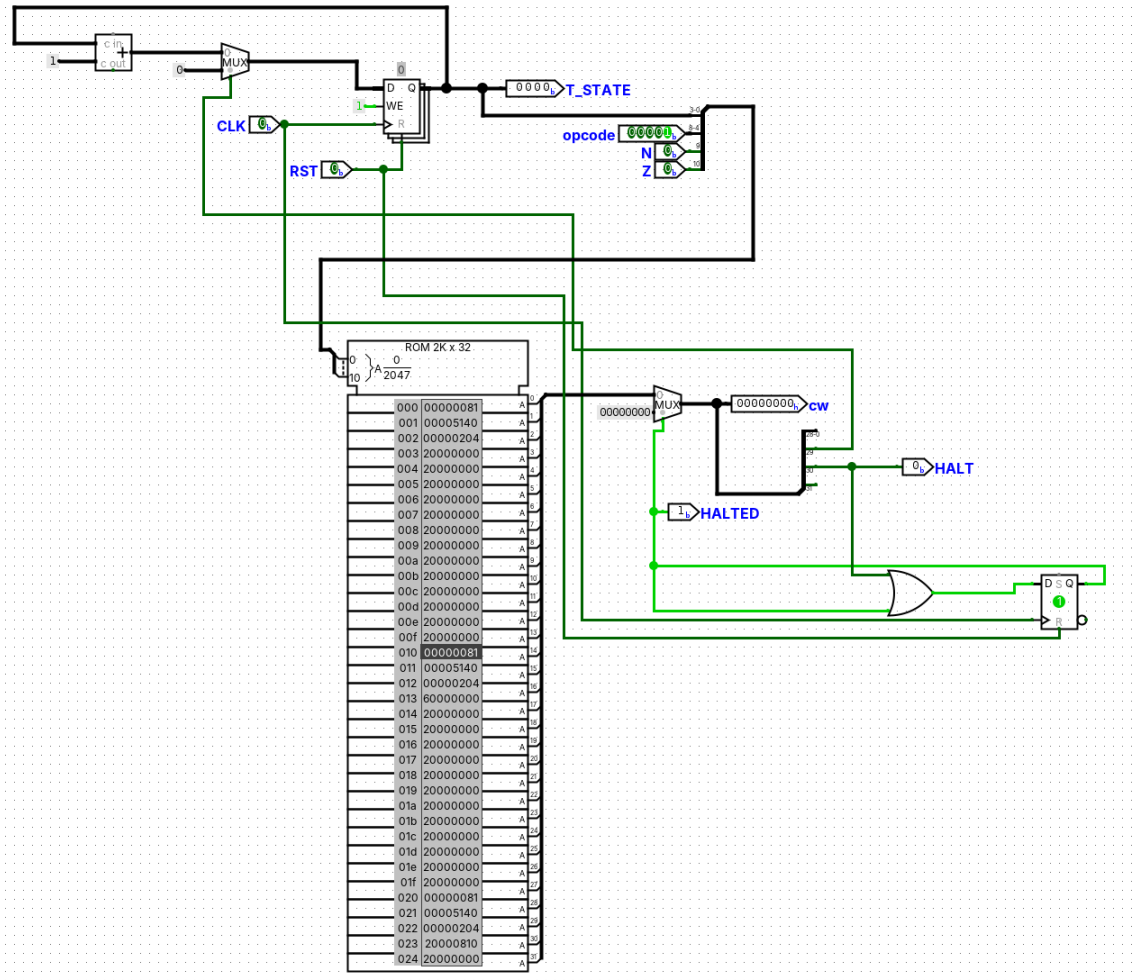


Figure 2. control_unit after t01: HALTED = 1, T_STATE = 0, cw masked to 0. The ROM shows the generated words: 081, 5140, 204 at T0-T2 of every opcode, 6000000 at HLT T3 (address 0x013).



Figure 3. Register file (R0 top to R7 bottom) of our circuit after HLT for ten test programs; every column equals the expected values in the table above (e.g. t03 R0 = abcd, t05 R3 = 8000, t09 R4 = 4000, p1 R3 = 2221).

6. Faults encountered and how they were found

Fault	Symptom, diagnosis and fix
Register file could not read R7 on port B.	SUB and SLT returned E (conflict) in bit 0 whenever R7 was the second operand, while R2/R3 worked. Minimal programs isolated it to port B with rb=7: the wire from R7 to its port-B tri-state buffer in regfile8x16 ended at (830,1020), 10 units short of the buffer input at (830,1030), so qb floated. Fixed by completing the wire; test t20 now reads every register through both ports.
Every fetch returned the previous instruction.	Registers were right in straight-line code but PC finished 2 too high. A cycle trace showed MDR capturing the word of the old MAR: Logisim's RAM reads on the clock edge by default, the same edge at which MDR samples. Setting Asynchronous read = Yes on both byte RAMs made the output follow MAR within the state.
TH4 ALU operands passed through controlled inverters.	ALU A was NOT(bus) gated by ACC_LOAD and B was NOT(qa) gated by a format decoder, so operands were inverted or floating, and A formed a loop through the bus. Replaced by A = mux(qa, PC) and B = mux(qb, IMM) with ALU_A_SEL and ALU_B_SEL; ACC became a real register16. Also fixed: qa and qb shorted by a wire at (1200,220)-(1200,240), ra_enb/rb_enb floating, ALU OPCODE unconnected.
T-state counter stuck at T1.	Headless run of control_unit with opcode HLT: cw went 0x81, 0x5140, then stayed 0x5140. The +1 adder's input came from a stray input pin 'Q' instead of the T_STATE register output, so it always computed 0+1. Fixed by wiring the register output back to the adder; HLT then halts after T3 and ADD returns to T0.
RA mux select unconnected.	After adding the RA index mux, a netlist check showed its 2-bit select connected to nothing, which would leave port A undefined for every instruction. Added the RA_SEL pin.
Every program using LIH failed after integration.	LIL-only and register tests passed, but any constant built with LIL+LIH ended as xxxx. Test vectors on our imm_unit gave {AB, UU} for IMM_SEL=6 and UUU0 for IMM_SEL=4: the QA and rel11 input pins sat 10 units away from their tunnels. Wired both; imm_unit then passed 210/210 vectors and all 13 programs passed.
opcode wire ran along the datapath's output edge.	In tara_cpu the opcode wire went down x=1730, the column of the datapath's output ports; any further output would have been shorted onto it. Rerouted away from the box (preventive, found while reviewing the netlist).

CS2310 Take-home 6: Memory, branches, advanced instructions, integration

Group 38 | Roll number: **CS25B035** | Logisim-Evolution 4.1.0 | G38_th6_tara.circ, G38_th6_microcode_gen.py, G38_th6_control_rom.hex, G38_th6_tests/

Summary. The TH5 processor is extended with the eleven remaining graded instructions: LDW, STW, LDB, STB, JMP, BZ, BN, CALL, RET, PUSH and POP. Only two pieces of hardware were added: a **BYTE_MODE** byte-lane unit inside ram_word (byte read with zero-extension, and a write enable per byte bank) and a **register-file write-index multiplexer** that can force the destination to R6 or R7. Everything else is microcode. All 26 graded instructions pass their own tests, the documented boundary cases and the regression suite; all six integration programs run to HLT with registers, PC and all 2048 bytes of memory identical to TARA Studio (27/27 programs). The generator passes 7/7 consistency checks. Memory, branch and advanced pools claimed: 57/57.

1. Interface table (subcircuits changed in TH6; ports exactly as labelled in the file)

Subcircuit	Inputs	Outputs	Change in TH6
ram_word	addr[16], din[16], mem_read, mem_write, clk, **BYTE_MODE**	dout[16]	byte read path (A0 mux + zero-extend), per-bank write enables, high-bank data mux
tara_datapath	CLK, RST, PC_OUT, IR_OUT, MDR_OUT, IMM_OUT, RF_OUT, ALU_OUT, RAM_OUT_LD, MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_load, RF_WRITE, PC_EN, PC_LOAD, mem_read, mem_write, ALU_select[5], ALU_A_SEL, ALU_B_SEL, status_we, IMM_SEL[3], RA_SEL[2], **RF_IDX_SEL[2]**, **BYTE_MODE**	opcode[5], Z, N, PC[16], IR[16]	write-index mux (RD, 6, 7, RD) in front of regfile rd; BYTE_MODE to ram_word
tara_cpu (main)	RST; Clock CLK	-	new 23-end control-word splitter; IR_OUT = constant 0; ACC_load = ALU_OUT
control_unit	CLK, RST, Z, N, opcode[5]	cw[32], T_STATE[4], HALTED, HALT	unchanged; ROM reloaded with TH6 image
imm_unit	imm8[8], off5[5], rel8[8], rel11[11], QA[16], IMM_SEL[3]	IMM[16]	unchanged; its sext(off5), rel8x2, rel11x2 and const 2 forms are now used

All other subcircuits (ALU, tara_decoder, regfile8x16, status_reg, PC_MAR_MDR_IR, register16, adders, shifters, comparators) are unchanged from TH5; their ports are listed in the TH5 report.

2. Control word, TH6 layout (32 bits, all used)

Bits	Field	Meaning	Bits	Field	Meaning
0	PC_OUT	PC → bus	12	mem_read	memory output enable
1	MDR_OUT	MDR → bus	13	mem_write	M[MAR] ← MDR (byte or word)
2	IMM_OUT	imm_unit → bus	14-18	ALU_select	ALU opcode
3	RF_OUT	port A (qa) → bus	19	ALU_A_SEL	ALU A: 0 qa, 1 PC
4	ALU_OUT	ALU → bus (also loads ACC)	20	ALU_B_SEL	ALU B: 0 qb, 1 IMM
5	RAM_OUT_LD	memory → bus	21	status_we	latch Z N C V
6	MAR_LOAD	MAR ← bus	22-24	IMM_SEL	0 zext8 1 sext8 2 sext5 3 rel8x2 4 rel11x2 5 const2 6 LIH 7 zero
7	MDR_LOAD	MDR ← bus	25-26	RA_SEL	port A: 0 IR[7:5], 1 IR[10:8], 2 R6, 3 R7
8	IR_LOAD	IR ← bus	27-28	RF_IDX_SEL	write index: 0 IR[10:8], 1 R6, 2 R7
9	RF_WRITE	R[index] ← bus	29	STEP_RESET	next edge: T-state ← 0
10	PC_EN	PC update (PC+2 if PC_LOAD=0)	30	HALT	stop, cw forced to 0
11	PC_LOAD	with PC_EN: PC ← bus	31	BYTE_MODE	ram_word byte access, lane = addr[0]

3. Design decisions

Decision	Justification
Byte access with per-bank write enables (no read-modify-write).	ram_word already holds two 8-bit RAMs (bottom = high byte, top = low byte). In byte mode only the bank selected by A0 = addr[0] is written: top Store = mem_write AND (NOT BYTE_MODE OR A0), bottom Store = mem_write AND (NOT BYTE_MODE OR NOT A0), built with negated OR-gate inputs. The bottom bank's data input takes din[7:0] in byte mode, so the low byte of Rs reaches either lane. The neighbouring byte is never rewritten, and STB needs 3 execute states, not 5.
LDB zero-extends inside ram_word.	In byte mode dout = zext(A0 ? low bank : high bank); otherwise dout = {high, low}. LDB therefore uses the same MDR path as LDW and only differs by BYTE_MODE at T4.
Big-endian, word accesses aligned.	M[even] is the high byte, as in TARA Studio. Word accesses use addr[15:1] and ignore addr[0]; all test programs access words at even addresses.
Effective address on the ALU.	EA = Rb + sext(off5): port A reads IR[7:5], ALU B = imm_unit form 2, result goes over the bus into MAR (T3). off5 is sign-extended, so -16 ... +15 (tested with -16, -1, +14).
PC-relative reference point is the incremented PC.	Fetch already leaves PC = address + 2, so JMP/BZ/BN/CALL compute PC + sext(rel) x 2 with ALU_A_SEL = 1 (PC) and IMM_SEL = 3 or 4 (the x2 is a zero wired below the field). This matches the assembler and TARA Studio.
BZ / BN in two states via status_reg.	T3: Rt + 0 through the ALU (port A reads IR[10:8], B = constant 0) with status_we, so Z and N describe Rt. Rt + 0 is used instead of Rt OR Rt because port B is wired to IR[4:2]. T4 is addressed by the latched Z (BZ) or N (BN): taken = PC ← PC + rel8x2, not taken = STEP_RESET only. The words differ only in that bit (generator check 5). BZ never depends on an earlier instruction (t14).
CALL writes the link first.	T3: R6 ← PC (PC_OUT, RF_WRITE, RF_IDX_SEL = 1); T4: PC ← PC + rel11x2. RET: port A forced to R6 (RA_SEL = 2), RF_OUT, PC_EN + PC_LOAD.
Stack: PUSH decrements then stores; POP loads then increments.	PUSH: R7 ← R7 - 2 (ALU SUB, const 2, RF_IDX_SEL = 2), MAR ← R7, MDR ← Rs, M[MAR] ← MDR. POP: MAR ← R7, MDR ← M, Rd ← MDR, R7 ← R7 + 2. PUSH R7 stores the decremented R7; POP R7 gives M[R7] + 2. Both equal TARA Studio (t17).
Control word re-laid out to fit 32 bits.	TH6 needs RF_IDX_SEL (2) and BYTE_MODE (1). IR_OUT was never used by any instruction and is tied to 0; ACC_load is tied to ALU_OUT (ACC is observational). STEP_RESET and HALT keep bits 29/30, so control_unit needed no change, only a new ROM image.

Decision	Justification
Unimplemented opcodes HALT.	MUL (12) and opcodes 27-31 get HALT at T3 and change no state (check 6, test t21).
Memory size.	ram_word addresses 32 K words; TARA has 2 KB. Programs use 0x000-0x7FF, so behaviour is identical; addresses above 0x7FF are not wrapped (documented limitation, never used by the tests).

4. TH6 microprograms (T0-T2 fetch: 0x00000041, 0x000014A0, 0x00000102)

Instr.	T3	T4	T5	T6
LDW	MAR ← Rb+sext(off5) 0x00924050	MDR ← M[MAR] 0x000010A0	Rd ← MDR, step-reset 0x20000202	
STW	MAR ← EA	MDR ← Rs (RA_SEL=1) 0x02000088	mem_write, step-reset 0x20002000	
LDB	MAR ← EA	MDR ← zext byte (BYTE_MODE) 0x800010A0	Rd ← MDR	
STB	MAR ← EA	MDR ← Rs	mem_write + BYTE_MODE 0xA0002000	
JMP	PC ← PC+rel11x2 0x211A4C10			
BZ / BN	Rt+0, status_we 0x03F24000	Z / N = 1: PC ← PC+rel8x2 0x20DA4C10; else 0x20000000		
CALL	R6 ← PC 0x08000201	PC ← PC+rel11x2 0x211A4C10		
RET	PC ← R6 0x24000C08			
PUSH	R7 ← R7-2 0x17528210	MAR ← R7 0x06000048	MDR ← Rs 0x02000088	M ← MDR 0x20002000
POP	MAR ← R7 0x06000048	MDR ← M 0x000010A0	Rd ← MDR 0x00000202	R7 ← R7+2 0x37524210

Generator log (G38_th6_microcode_log.txt):

```
TARA microcode generator, TH6 — 26 instructions: NOP HLT MOV LIL LIH ADD SUB ADDI AND OR XOR NOT SHL SHR SLT LDW STW LDB ST
ROM 2048 x 32 bits

[PASS] 1. no control word selects more than one bus source
[PASS] 2. every instruction eventually asserts STEP_RESET or HALT
[PASS] 3. mem_read and mem_write are never asserted together
[PASS] 4. write enables are never asserted without valid source data
[PASS] 5. conditional-branch words differ only in the relevant condition signal
[PASS] 6. no undefined opcode can modify architectural state (they HALT)
[PASS] 7. T0-T2 hold the common fetch sequence for every opcode

7/7 checks passed
```

5. Test evidence

Every program is assembled with the TARA Studio assembler, written into ram_word's two byte banks, and run on our circuit with Logisim's own simulator until HALTED = 1; the same program runs on TARA Studio. PASS means R0-R7, PC and all 2048 memory bytes are identical. Per-instruction traces of the six integration programs, recorded on our circuit, are in G38_th6_tests/traces; TARA Studio logs of every program are in reference_logs; screenshots of both for every integration program are in screenshots.

Program	Tests	R0 ... R7 after HLT (ours = TARA Studio)	PC	Result
t11	LDW/STW, off5 -16 and +14	0000 0410 1234 1234 0410 1234 0000 0410	001C	PASS
t12	LDB/STB, STB at odd and even address	00AB 0400 1234 0012 0034 99EE 12EE EE00	0022	PASS
t13	JMP forward and back	0000 0001 0002 0003 0000 0000 0000 0000	0012	PASS
t14	BZ taken / not taken, BZ on FFFF	0000 0000 FFFF 0000 0000 0004 0000 0000	0024	PASS
t15	BN on positive, zero, 8000	FFFF 0005 8000 0000 0003 0000 0000 0000	001E	PASS
t16	CALL/RET, nested with PUSH R6	0000 0052 0000 0000 0000 0014 000C 05F0	000E	PASS
t17	PUSH/POP, PUSH R7, POP R7	05F0 0011 AA22 AA22 0011 05EE 05EE 0013	0020	PASS
t18	POP at top of memory (07FE)	0000 0077 0077 0800 0000 0000 0000 0800	0012	PASS
t19	regression mix of all classes	0406 0000 0015 002A 002A FFC0 002A 05E6	002C	PASS
p1	register addition	0000 1234 0FED 2221 0000 0000 0000 0000	000C	PASS
p2	memory subtraction (M[404]=06A4)	0400 012C 07D0 012C 06A4 0000 0000 0000	001A	PASS
p3	parity of B6D5	0001 0000 0000 0001 0000 B6D5 0000 0000	001A	PASS
p4	123 x 45 by repeated addition	0000 007B 0000 159F 0000 0000 0000 0000	0010	PASS
p5	sum of 16-bit array = 1194	0400 0410 0000 1194 0000 007D 0000 0000	0030	PASS
p6	max of signed array = 05DC (CALL, PUSH, POP)	0400 040E 0000 05DC 0384 0000 0410 05F0	004C	PASS

Also passing on the same file: t01-t10, t20 (all 15 core instructions) and t21 (MUL halts): 27/27.

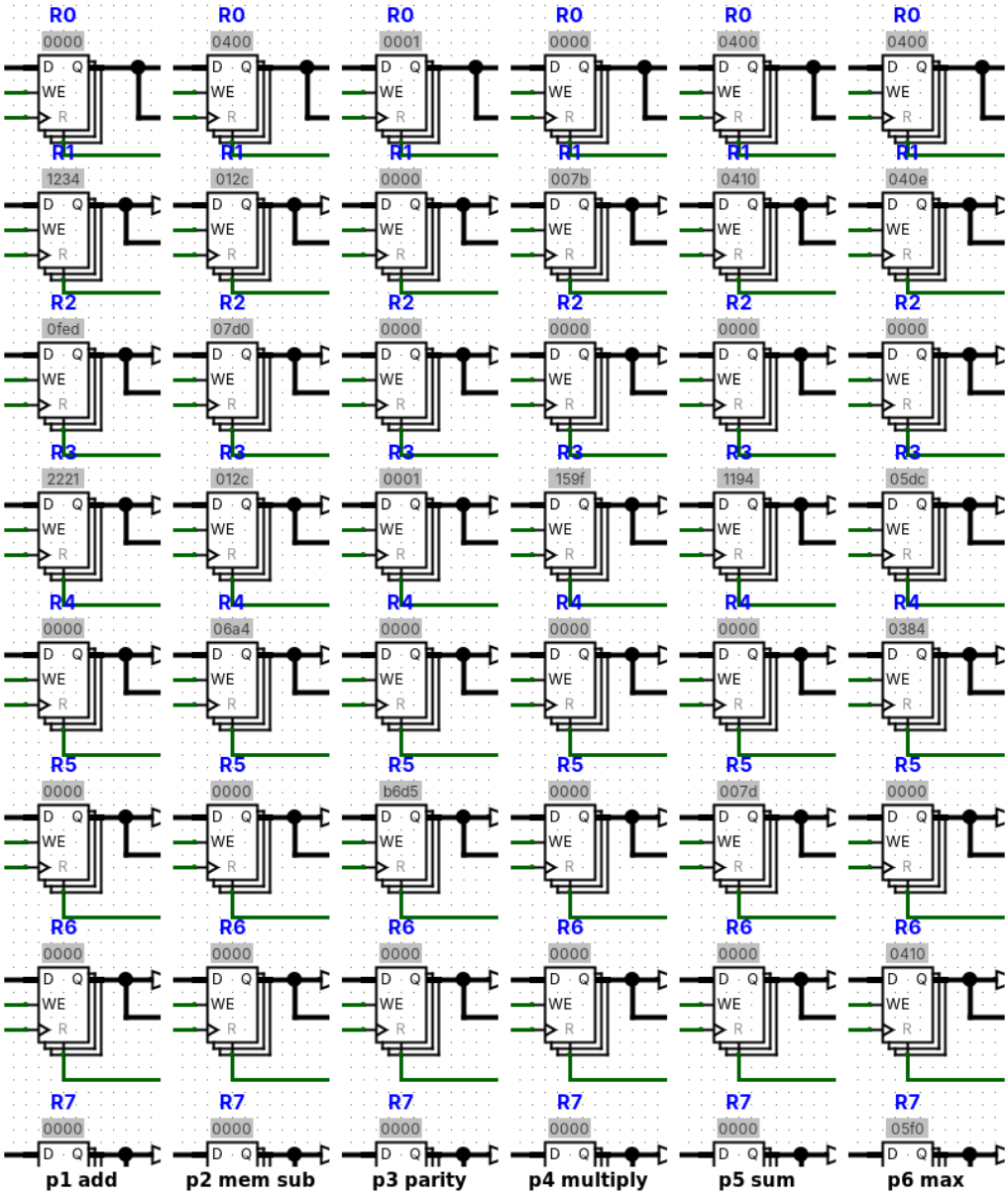


Figure 1. Register file of OUR circuit in Logisim after HLT for the six integration programs (R0 top to R7 bottom). p4: R3 = 159f = 123 x 45; p6: R3 = 05dc = 1500, R7 back at 05f0 after the CALL/PUSH/POP loop.

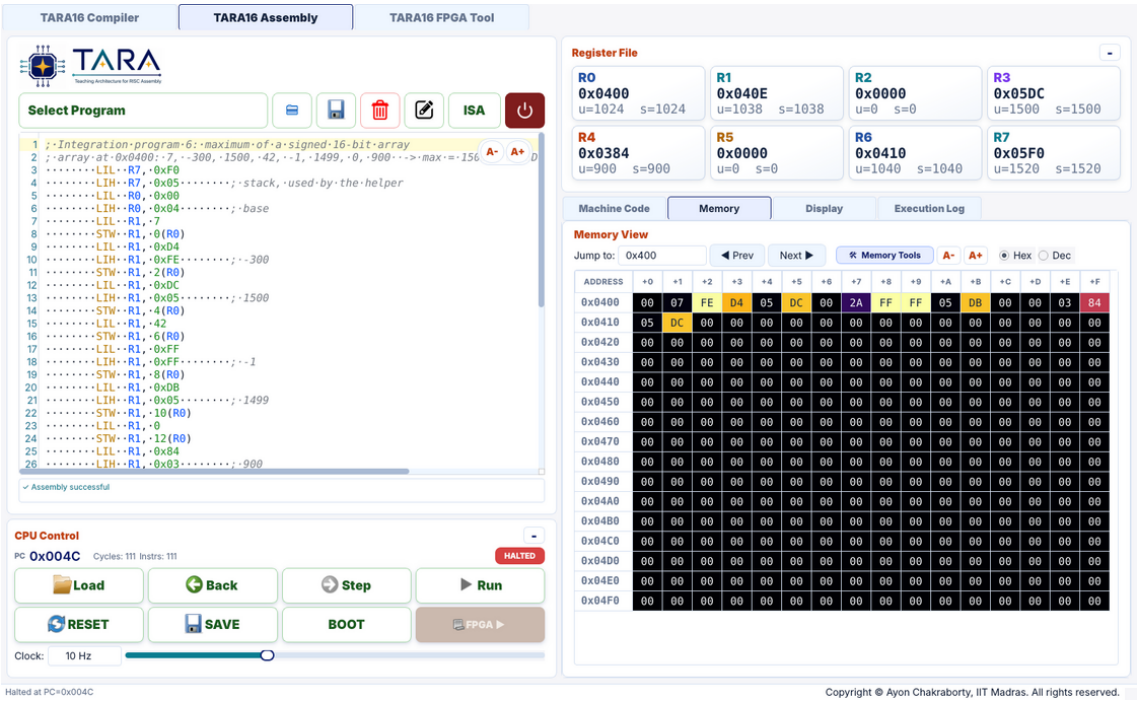


Figure 2. TARA Studio after p6 (HALTED, PC = 0x004C): same registers as Figure 1, memory at 0x400 shows the array and the result 05DC at 0x410. Screenshots for p1-p6 are in G38_th6_tests/screenshots.

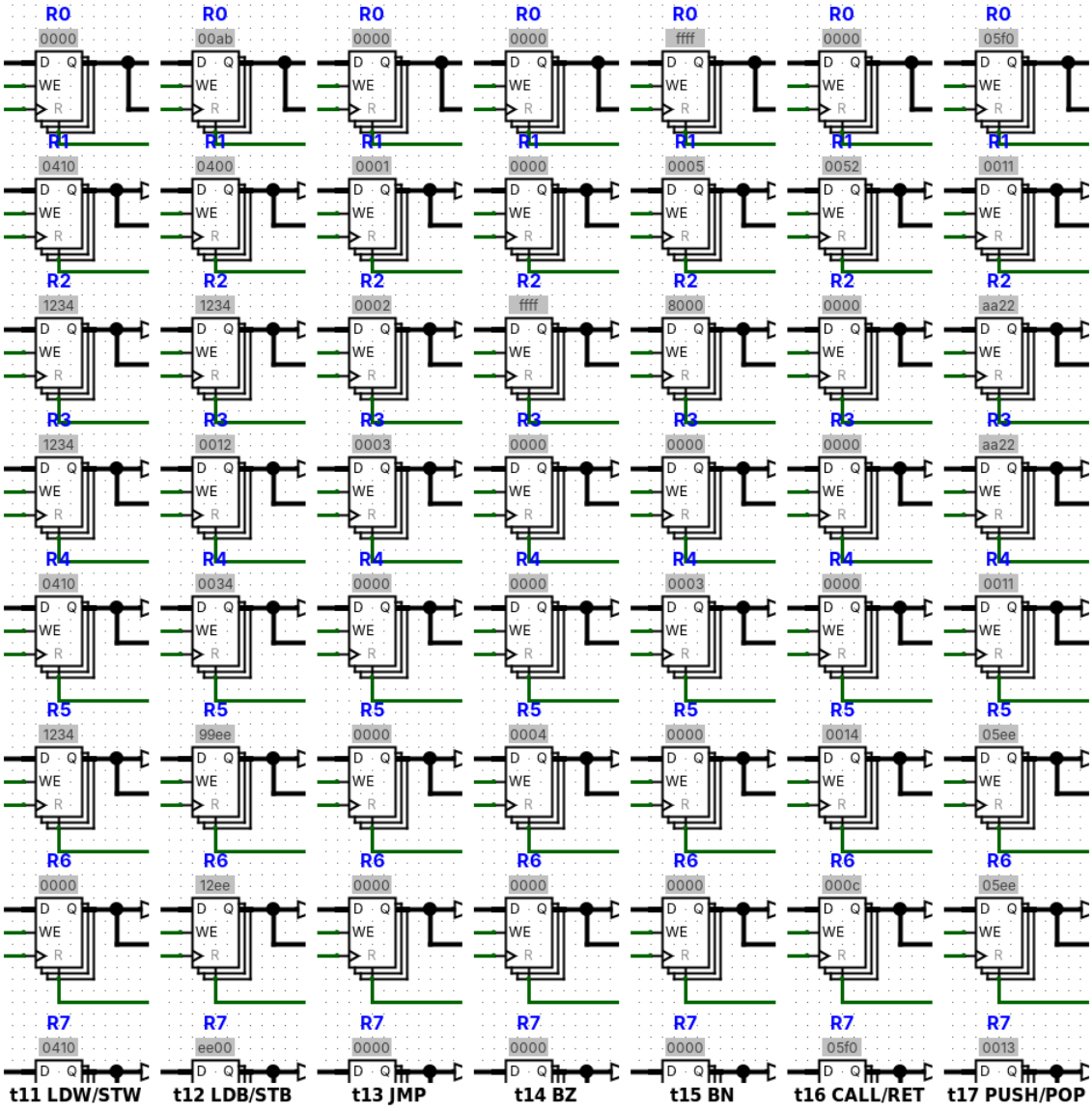


Figure 3. Our register file after the TH6 instruction tests. t12: R3 = 0012 / R4 = 0034 (LDB high and low byte), R6 = 12ee (STB at odd address kept the 12); t16: R6 = 000c link; t17: R7 = 0013 (POP R7).

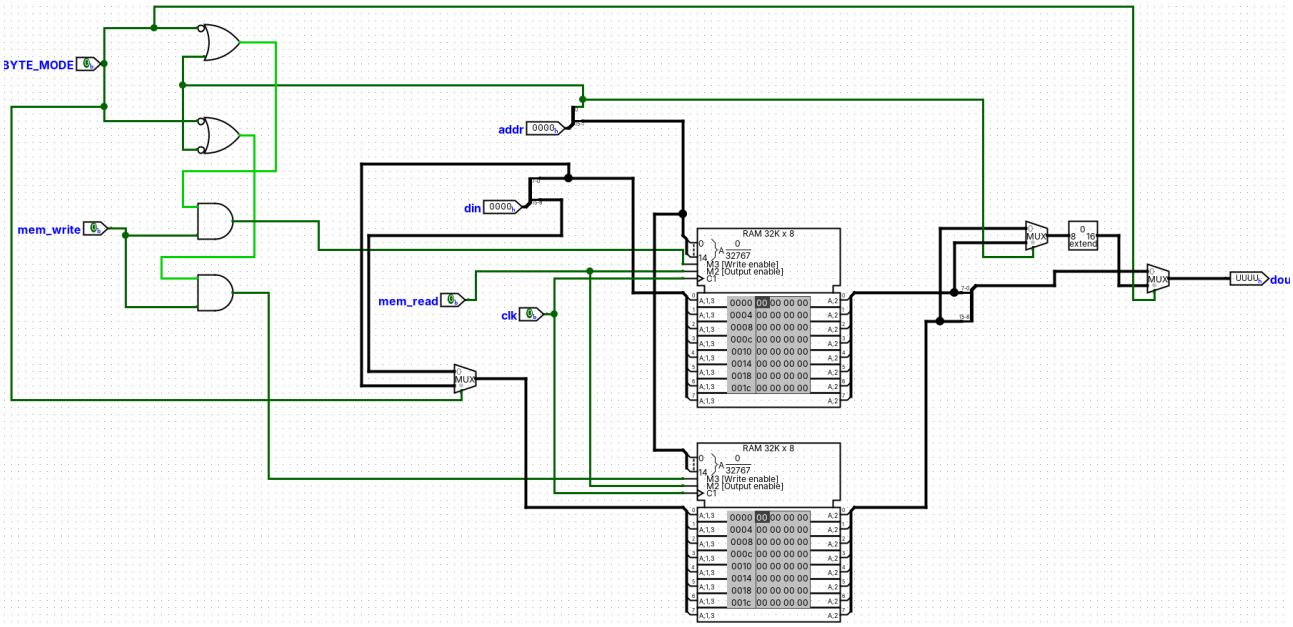


Figure 4. ram_word with the byte lanes: OR gates with negated inputs and AND gates give the two Store enables, the left mux feeds din[7:0] to the high bank in byte mode, the A0 mux + zero extender + right mux form dout.

6. Faults encountered and how they were found

Fault	Symptom, diagnosis and fix
Write-index mux select left floating.	After adding the RF write-index mux, a netlist check of tara_datapath showed its 2-bit select connected to nothing: there was no RF_IDX_SEL pin. Every register write would have gone to an undefined index. Added the RF_IDX_SEL input pin and connected it to splitter end 20 (bits 27-28) in tara_cpu.
Control word did not fit in 32 bits.	The TH5 word used 31 bits; RF_IDX_SEL and BYTE_MODE needed three more. Found while planning the layout. Solved by removing IR_OUT (no microprogram ever drives IR onto the bus; tied to 0) and deriving ACC_load from ALU_OUT, keeping STEP_RESET/HALT at bits 29/30 so control_unit stayed unchanged.
TH5 ROM image still loaded after the layout change.	The rewired tara_cpu was checked by comparing the ROM contents in the file with the generator output: the first words were still 0x81, 0x5140, 0x204 (TH5 fetch). With the new layout those bits mean other signals, so even fetch would fail. Reloading G38_th6_control_rom.hex gave 0x41, 0x14A0, 0x102 and an exact match over all 2048 words.
Main circuit set to the empty 'main'.	The saved file opened on an empty canvas because the main circuit was the old empty 'main', so a grader starting the simulation would see nothing. Set tara_cpu as main circuit.
Byte-lane test appeared to fail.	The first stand-alone test of ram_word returned 0000 for every read. Running the same test on the TH5 ram_word gave the same zeros, which located the fault in the test itself: its first write fell on the initial clock edge and was lost. With an idle first cycle the TH5 RAM passed the word cases (and failed the byte ones, as expected) and the TH6 RAM passed all seven cases, including STB at odd and even addresses with the neighbour byte preserved.